

# Báo cáo hằng tuần

---

## Thông tin chung

- Mã nhóm: **06**
- Chủ đề: **T02 — Web Automation Testing**
- Thời gian: **06/07 - 11/07/2026**

## Các công việc đã hoàn thành trong tuần này

---

### 23127378 – Nguyễn Gia Huy

- Tìm hiểu chi tiết công cụ Mabl về cài đặt và sử dụng cơ bản
- Tìm hiểu cách thực thi cách một kịch bản test script của công cụ Mabl
- Tìm hiểu AI test automation, các ứng dụng AI vào software testing
- Tài liệu sinh viên đã dùng để tham khảo
  - [Playlist YouTube hướng dẫn Mabl](#)
  - [Playlist YouTube AI Test Automation](#)
  - [Trang chủ Mabl](#)
- Artifact của sinh viên:
  - [Tóm tắt nội dung video AI test automation](#)

### 23127379 – Thái Minh Huy

- Tìm hiểu chi tiết công cụ Playwright về cài đặt và sử dụng cơ bản
  - Cách cài đặt công cụ Playwright ở trên hệ điều hành với công cụ phải có là NodeJS và npm để cài đặt
  - Cách sử dụng các câu lệnh của công cụ Playwright để chạy test
  - Cách sử dụng các nút được để chạy test thông qua script example.spec.js của công cụ Playwright.
- Tìm hiểu cách thực thi cách một kịch bản test script của công cụ Playwright
  - Cách sử dụng tùy biến môi trường khi chạy kịch bản test của Playwright thông qua file playwright.config.js
  - Coi Tutorial và cách viết một testcase trên một Functional Requirements và chạy thử kịch bản test với các câu lệnh npx playwright tests,... Sau đó sử dụng CodeGen, một tính năng của Playwright để mà giả lập các thao tác mà mình thực hiện rồi sinh ra những prototype script test đầu tiên.
- Ngoài ra, tìm hiểu cách viết kịch bản test với Playwright với hai cách khác nhau, một là sử dụng với CSS/XPath dù không cần đụng vào mã nguồn nhưng rất dễ xảy ra tình trạng flaky test (nghĩa là cùng 1 kịch bản test mà lần đầu pass, lần sau fail). Hai là sử dụng dưới dạng là data-testid sẽ cần phải bổ sung metadata trên mã nguồn ReactJS nhưng đảm bảo testcase ổn định
- Tài liệu sinh viên đã dùng để tham khảo
  - [Tài liệu chính thức của Playwright](#)
  - [Video hướng dẫn Playwright](#)

- Một kịch bản kiểm thử mà sinh viên sử dụng là data-testId (đây là kịch bản sinh viên sử dụng khi đi phòng vấn technical)
- Artifact của sinh viên:
  - Cài đặt Playwright
  - Các câu lệnh sử dụng của Playwright
  - Kịch bản kiểm thử sinh viên viết và có sử dụng A.I hỗ trợ để chạy Playwright tests Đây là những kịch bản đầu tiên để tìm hiểu, chưa phải là bản cuối cùng

## 23127396 – Lương Linh Khôi

- Tìm hiểu tổng quan về công cụ Mabl và quy trình kiểm thử tự động
  - Tìm hiểu khái niệm kiểm thử tự động (Automated Testing) trên nền tảng Mabl và cách công cụ hỗ trợ kiểm thử End-to-End (E2E) cho ứng dụng Web.
  - Tìm hiểu giao diện làm việc của Mabl, bao gồm Workspace, Application, Plan, Journey và Test Output.
  - Tìm hiểu cách Mabl sử dụng trí tuệ nhân tạo (A.I.) để tự động nhận diện phần tử trên giao diện, giúp giảm thiểu việc cập nhật testcase khi giao diện thay đổi.
- Tìm hiểu cách tạo và thực thi một kịch bản kiểm thử trên Mabl
  - Cách tạo một Journey mới và ghi lại (Record) các thao tác của người dùng trên trình duyệt.
  - Cách thêm các bước kiểm thử như Click, Type, Assert, Wait, Navigate và Validation vào trong Journey.
  - Thực hành chạy thử (Run) các kịch bản kiểm thử và theo dõi kết quả thực thi thông qua báo cáo của Mabl.
- Tìm hiểu các tính năng hỗ trợ kiểm thử của Mabl
  - Tìm hiểu khả năng tự động phát hiện sự thay đổi giao diện (Visual Testing) và các lỗi phát sinh trong quá trình kiểm thử.
  - Tìm hiểu cách sử dụng biến (Variables), dữ liệu kiểm thử (Test Data) và tham số hóa (Parameterization) để tái sử dụng kịch bản kiểm thử.
  - Tìm hiểu cơ chế quản lý nhiều môi trường kiểm thử (Development, Staging, Production) và cách cấu hình môi trường khi chạy Journey.
- Tài liệu tham khảo
  - Tài liệu chính thức của Mabl: <https://help.mabl.com/>
  - Video chính thức của Mabl trên YouTube: <https://www.youtube.com/@mablHQ>
- Artifact
  - Tài liệu ghi chú quá trình tìm hiểu và cài đặt Mabl.  
[https://docs.google.com/document/d/1XDG7C7cxh\\_IJHwr00-oZAHNjD97fTNCj3gE5ECpQEA/edit?usp=sharing](https://docs.google.com/document/d/1XDG7C7cxh_IJHwr00-oZAHNjD97fTNCj3gE5ECpQEA/edit?usp=sharing)
  - Kết quả thực thi (Run Results) và báo cáo kiểm thử của các Journey đã chạy thử.  
[https://docs.google.com/document/d/1iuNXLpSVYHzBI5G1H\\_5krTFVU9pUj6p6z0FTJ3eIUZc/edit?usp=sharing](https://docs.google.com/document/d/1iuNXLpSVYHzBI5G1H_5krTFVU9pUj6p6z0FTJ3eIUZc/edit?usp=sharing)

## 23127449 - Nguyễn Tấn Phát

- Thiết kế các skill và workflow cho web automation testing.
- Setup cấu trúc project theo hướng kết hợp với AI Agent để hỗ trợ cho quá trình sinh ra các kịch bản và các script test.
- Các artifact có thể xem tại branch [web-automation-testing](#) của repository sau: [phatnguyen975/eshop-sut](#). Trong đó, file [skill-execution-guide.md](#) hướng dẫn cách sử dụng skill và workflow các bước để tạo ra các kịch bản test, viết script test và tiến hành chạy để kiểm thử SUT.

## Khai báo sử dụng AI

### 23127378 – Nguyễn Gia Huy

- Gemini 3.1 Pro, Google. Truy cập 7h45 ngày 11/07/2026
  - Mục đích: tạo một tài liệu Hướng dẫn sử dụng (User Guide) toàn diện, dễ hiểu về nền tảng kiểm thử tự động Mabl dành cho người mới bắt đầu
  - Prompt:

#### Nhiệm vụ:

Bạn là một chuyên gia về Test Automation và Technical Writer có nhiều kinh nghiệm. Hãy viết một "Hướng dẫn sử dụng (User Guide) toàn diện cho người mới bắt đầu" về nền tảng kiểm thử tự động Mabl.

#### Mục tiêu:

Giúp người dùng mới (bao gồm cả QA/Tester không chuyên code và Automation Engineer) hiểu Mabl là gì, cách thức hoạt động của nó, và làm thế nào để tạo, chạy, và bảo trì các bài test cơ bản.

#### Giọng văn:

Chuyên nghiệp, rõ ràng, dễ hiểu, sử dụng các thuật ngữ đúng chuyên ngành Kiểm thử Phần mềm (nhưng cần có giải thích ngắn gọn cho người mới). Sử dụng định dạng Markdown (tiêu đề, in đậm, gạch đầu dòng, blockquote) để cấu trúc rõ ràng.

Nội dung yêu cầu cấu trúc theo các phần sau:

#### 1. Giới thiệu về Mabl:

Mabl là gì? (Nhấn mạnh đây là nền tảng Agentic Testing, Low-code, AI-native).

Tại sao nên dùng Mabl? (Nêu bật các điểm mạnh như: Auto-healing, không cần quản lý hạ tầng – fully managed in the cloud, dễ dàng tích hợp CI/CD).

#### 2. Các khái niệm cốt lõi (Key Concepts):

Giải thích ngắn gọn: Workspace, Application, Environment, Plan, Test.

#### 3. Bắt đầu (Getting Started):

Cách cài đặt mabl Desktop App / Browser Extension (mabl Trainer).

Cách thiết lập Application và Environment đầu tiên.

#### 4. Tạo bài test giao diện Web (Creating a Browser Test):

Cách 1: Sử dụng mabl Agent (AI Prompt) – Hướng dẫn cách viết prompt để AI tự động sinh test outline (ví dụ: mô tả user journey, cung cấp URL).

Cách 2: Sử dụng mabl Trainer (Record & Playback) – Hướng dẫn cách thao tác trên trình duyệt để ghi lại các bước (click, nhập text) và cách thêm Assertions (xác nhận kết quả).

#### 5. Các tính năng nâng cao cần biết:

Auto-healing: Cách mabl tự động cập nhật test khi UI thay đổi.

Visual Testing: Cách mabl phát hiện sự thay đổi về giao diện (không chỉ kiểm tra code).

Tích hợp (Integrations): Liệt kê ngắn gọn cách mabl chạy API test, chèn JavaScript snippet, hoặc tích hợp với CI/CD (GitHub Actions, Jira, Slack).

#### 6. Thực thi và Phân tích kết quả (Running & Analyzing):

Cách chạy test (Local run vs Cloud run).

Cách xem báo cáo: Phân tích log, ảnh chụp màn hình (screenshots), và thông tin chẩn đoán khi test thất bại (Failure Analysis).

#### 7. Thực hành tốt nhất (Best Practices):

Đưa ra 3–4 lời khuyên để viết test ổn định, không bị flaky (chập chờn) trên Mabl.

Lưu ý cho AI: Xin đừng tạo ra thông tin không có thật. Hãy dựa trên các tính năng hiện tại của Mabl (như mabl Trainer, AI-native auto-healing, API/UI testing) tính đến thời điểm hiện tại.

#### ◦ Kết quả:

### ## 1. Giới thiệu về Mabl

Mabl là một nền tảng kiểm thử tự động thông minh (Intelligent Test Automation Platform) được thiết kế theo phương pháp **\*\*Low-code\*\*** (ít phải viết mã) và **\*\*AI-native\*\*** (tích hợp trí tuệ nhân tạo từ lõi). Mabl đặc biệt nổi bật với khái niệm **\*\*Agentic Testing\*\***, nơi các "đặc

vụ" AI (Agents) có thể hiểu ý định của bạn qua ngôn ngữ tự nhiên để tự động tạo và bảo trì các kịch bản kiểm thử.

**\*\*Tại sao nên dùng Mabl?\*\***

\* **\*\*Auto-healing (Tự phục hồi):\*\*** Khi ứng dụng của bạn thay đổi giao diện (đổi tên class CSS, thay đổi cấu trúc HTML), AI của Mabl sẽ tự động nhận diện và cập nhật kịch bản test mà không làm fail bài test một cách vô lý.

\* **\*\*Fully managed in the cloud (Quản lý hoàn toàn trên mây):\*\*** Bạn không cần cài đặt server, quản lý Selenium Grid hay lo lắng về cơ sở hạ tầng. Mọi thứ được thực thi trên môi trường đám mây của Mabl.

\* **\*\*Tiếp cận dễ dàng, sức mạnh to lớn:\*\*** Người không biết code có thể tạo test qua thao tác click chuột (Record & Playback), trong khi dân kỹ thuật có thể chèn thêm các đoạn mã JavaScript phức tạp hoặc gọi API.

\* **\*\*Tích hợp CI/CD liền mạch:\*\*** Dễ dàng gắn vào các quy trình phát hành phần mềm liên tục để phát hiện lỗi sớm.

---

## ## 2. Các khái niệm cốt lõi (Key Concepts)

Để sử dụng Mabl hiệu quả, bạn cần nắm vững hệ thống phân cấp tổ chức dữ liệu của nó:

\* **\*\*Workspace (Không gian làm việc):\*\*** Nơi chứa toàn bộ dữ liệu, bài test, và cấu hình của công ty hoặc dự án của bạn. Các thành viên trong team sẽ được mời vào đây để cộng tác.

\* **\*\*Application (Ứng dụng):\*\*** Sản phẩm phần mềm bạn đang kiểm thử (ví dụ: Website Thương mại điện tử A, App Quản lý nhân sự B).

\* **\*\*Environment (Môi trường):\*\*** Nơi ứng dụng đang chạy. Bạn có thể có nhiều môi trường cho một ứng dụng như *\*Development\**, *\*Staging\**, hoặc *\*Production\**.

\* **\*\*Plan (Kế hoạch chạy test):\*\*** Một tập hợp các bài Test được cấu hình để chạy cùng nhau trên một Environment cụ thể, theo một lịch trình nhất định (ví dụ: chạy mỗi đêm) hoặc được kích hoạt từ hệ thống CI/CD.

\* **\*\*Test (Kịch bản kiểm thử):\*\*** Tập hợp các bước cụ thể (click, nhập chữ, kiểm tra dữ liệu) để xác minh một tính năng của ứng dụng (ví dụ: Test quy trình Đăng nhập).

---

## ## 3. Bắt đầu (Getting Started)

Để bắt đầu hành trình với Mabl, bạn cần thiết lập công cụ và môi trường làm việc cơ bản.

**\*\*Cài đặt công cụ mabl Trainer:\*\***

Có hai cách để sử dụng mabl Trainer, nhưng **\*\*mabl Desktop App\*\*** luôn được khuyến nghị vì hỗ trợ đầy đủ các tính năng nâng cao (như test API, kiểm tra tải file).

1. Đăng nhập vào web app của Mabl, tìm menu **\*\*Download mabl Desktop App\*\*** và cài đặt lên máy tính của bạn (Windows/Mac).
2. (Tùy chọn) Cài đặt **\*\*mabl Browser Extension\*\*** trên Chrome nếu bạn chỉ muốn ghi lại thao tác test web cơ bản.

**\*\*Thiết lập Application và Environment đầu tiên:\*\***

1. Vào mục **\*\*Configuration\*\*** > **\*\*Applications\*\*** trên giao diện Mabl.
2. Nhấn **\*\*New application\*\*** và nhập tên ứng dụng của bạn.
3. Vào mục **\*\*Environments\*\***, tạo một môi trường mới (ví dụ: "Staging").
4. Liên kết Application với Environment vừa tạo bằng cách cung cấp **\*\*Base URL\*\*** (địa chỉ web chính của ứng dụng trên môi trường đó, ví dụ: `[https://staging.myapp.com](https://staging.myapp.com)`).

---

#### ## 4. Tạo bài test giao diện Web (Creating a Browser Test)

Bạn có hai phương pháp chính để tạo một kịch bản kiểm thử giao diện trong Mabl.

##### ### Cách 1: Sử dụng mabl Agent (AI Prompt)

Tính năng Agentic Testing cho phép bạn mô tả kịch bản bằng ngôn ngữ tự nhiên. AI sẽ tự động phân tích và sinh ra các bước test.

1. Tạo Test mới và chọn phương thức **\*\*Generate with AI\*\***.
2. Cung cấp URL của trang web bạn muốn kiểm thử.
3. Viết prompt (câu lệnh) mô tả chi tiết User Journey (Hành trình người dùng).
4. Ví dụ Prompt: *"Truy cập vào trang chủ, click vào nút 'Đăng nhập' ở góc phải. Nhập email là 'test@example.com' và mật khẩu '123456'. Bấm nút 'Submit'. Xác nhận rằng dòng chữ 'Chào mừng trở lại' xuất hiện trên màn hình."*
5. Mabl Agent sẽ tự động tạo ra bộ khung test (Test Outline). Bạn có thể xem lại, chỉnh sửa hoặc chạy thử để AI hoàn thiện các bước.

##### ### Cách 2: Sử dụng mabl Trainer (Record & Playback)

Đây là phương pháp truyền thống và trực quan nhất để tạo test bằng cách thao tác trực tiếp trên giao diện.

1. Tạo Test mới và chọn **\*\*Train a test\*\***. Ứng dụng mabl Desktop sẽ mở ra kèm theo cửa sổ trình duyệt và thanh công cụ mabl Trainer.
2. **\*\*Ghi lại thao tác (Record):\*\*** Bạn tương tác với trang web (click, cuộn trang, điền form) giống hệt như một người dùng thật. Mabl sẽ tự động ghi lại từng hành động thành các "Step".
3. **\*\*Thêm Assertions (Xác nhận):\*\*** Để đảm bảo bài test có ý nghĩa, bạn cần kiểm tra kết quả. Nhấn vào biểu tượng **\*\*dấu tích (Checkmark)\*\*** trên thanh Trainer, trở chuột vào một phần tử trên web (ví dụ: thông báo thành công) và thiết lập điều kiện (ví dụ: *\*Equals\**, *\*Contains\**, *\*Is Present\**).

#### 4. Lưu bài test và đóng Trainer.

----

#### ## 5. Các tính năng nâng cao cần biết

Mabl không chỉ dừng lại ở việc click và nhập liệu. Dưới đây là các tính năng biến nó thành công cụ mạnh mẽ:

- \* **Auto-healing (Tự phục hồi):** Bạn không cần can thiệp thủ công. Khi UI thay đổi nhẹ (đổi ID, thay đổi màu sắc, dịch chuyển vị trí), Mabl sử dụng nhiều chiến lược thu thập dữ liệu DOM để tìm ra phần tử đúng và tự động cập nhật bài test.

- \* **Visual Testing (Kiểm tra giao diện trực quan):** Mabl chụp lại ảnh màn hình ở mỗi bước. Nếu hôm nay nút "Mua hàng" vô tình bị một banner che khuất hoặc bị lỗi CSS làm đổi màu, tính năng Visual Testing sẽ giống lên cảnh báo cho bạn, dù code chức năng vẫn hoạt động.

- \* **Tích hợp (Integrations):**

- \* **API Testing:** Bạn có thể gọi API trực tiếp trong lúc chạy test UI để chuẩn bị dữ liệu (ví dụ: gọi API tạo tài khoản trước khi test chức năng đăng nhập trên web).

- \* **JavaScript Snippets:** Dùng cho các bài toán phức tạp như format ngày tháng, tính toán giá trị giỏ hàng.

- \* **CI/CD & Notifications:** Dễ dàng kích hoạt test qua GitHub Actions, GitLab CI. Kết quả fail có thể được bắn trực tiếp vào kênh Slack hoặc tự động tạo ticket trên Jira.

----

#### ## 6. Thực thi và Phân tích kết quả (Running & Analyzing)

Sau khi tạo test, việc chạy và đọc hiểu kết quả là kỹ năng quan trọng nhất.

**\*\*Cách chạy Test:\*\***

- \* **Local Run:** Chạy trực tiếp trên máy tính của bạn thông qua mabl Desktop App. Không tốn thời gian chờ cấp phát server, thích hợp để debug nhanh khi bạn vừa viết xong test.

- \* **Cloud Run:** Chạy trên hạ tầng của Mabl. Bạn có thể thiết lập Plan để chạy song song (Parallel) hàng chục bài test trên nhiều trình duyệt (Chrome, Firefox, Safari, Edge) cùng một lúc.

**\*\*Phân tích kết quả (Failure Analysis):**

Khi một bài test thất bại trên Cloud, Mabl cung cấp thông tin chẩn đoán rất sâu:

- \* **Screenshots (Ảnh chụp màn hình):** Xem lại chính xác màn hình trông như thế nào ở giây phút bị lỗi.

- \* **DOM Snapshots:** Xem cấu trúc HTML tĩnh tại thời điểm lỗi để kiểm tra xem phần tử bạn cần có thực sự tồn tại trong code hay không.

- \* **Network Activity:** Kiểm tra xem có API nào bị lỗi (ví dụ trả



về mã 404, 500) hoặc load quá chậm gây ảnh hưởng đến giao diện không.

\* **\*\*Logs:\*\*** Xem lại toàn bộ thông báo từ Console của trình duyệt.

---

## ## 7. Thực hành tốt nhất (Best Practices)

Để hệ thống test của bạn trên Mabl luôn xanh (Pass) và không bị chập chờn (Flaky), hãy áp dụng các nguyên tắc sau:

> **\*\*Viết test dựa trên góc nhìn của người dùng:\*\*** Đừng cố bám vào các bộ chọn (selectors) CSS phức tạp. Hãy tương tác với các thành phần dựa trên text hoặc vai trò (role) của chúng giống như cách một người dùng thực sự nhìn thấy trên màn hình.

\* **\*\*Tránh dùng "Hard Sleep":\*\*** Thay vì bắt bài test dừng cứng lại 5 giây để chờ trang tải, hãy sử dụng tính năng **\*\*Wait until\*\*** (Chờ cho đến khi) một phần tử cụ thể xuất hiện. Mabl vốn đã có cơ chế tự động chờ nội bộ rất tốt, hãy tận dụng nó.

\* **\*\*Mô-đun hóa bằng Reusable Flows:\*\*** Nếu tính năng "Đăng nhập" được dùng lại trong 50 bài test khác nhau, hãy tạo một Reusable Flow (Luồng dùng chung) cho nó. Khi quy trình đăng nhập thay đổi, bạn chỉ cần sửa ở một nơi duy nhất.

\* **\*\*Kiểm thử hướng dữ liệu (Data-driven Testing):\*\*** Tránh việc gán cứng (hardcode) dữ liệu vào bài test (ví dụ: "Nguyễn Văn A"). Hãy sử dụng **\*\*Variables\*\*** (Biến) và **\*\*Datatables\*\*** để chạy cùng một bài test với nhiều tập dữ liệu khác nhau (ví dụ: test đăng nhập với 5 tài khoản có quyền hạn khác nhau).

\* **\*\*Làm sạch dữ liệu (Teardown):\*\*** Nếu bài test của bạn tạo ra dữ liệu mới (ví dụ: tạo một đơn hàng), hãy chắc chắn rằng có các bước hoặc API được gọi ở cuối bài test để xóa đơn hàng đó, giúp môi trường luôn sạch sẽ cho các lần chạy sau.

- Nhận xét: Kết quả cho ra không được tốt lắm, chưa hướng dẫn đầy đủ các kịch bản sử dụng chi tiết làm cho người đọc khó hiểu

## 23127379 – Thái Minh Huy

- Gemini 3.1 Pro, Google. Truy cập 11h16 ngày 10/07/2026
  - Mục đích: Sinh kịch bản kiểm thử đầu tiên của FR02
  - Prompt:

### 6. Study Milestones (Stage S3 internal checklist)

1. Set up Playwright + npm; verify with the EShop login spec from FR-02.

2. Cover three flows: Login + logout (FR-02), Add-to-Cart (FR-07), Checkout (FR-08).

Để hiểu hơn, bạn hãy hướng dẫn tôi học và hiểu sâu tool này với những flow này đi. Hãy đi từ FR-02 trước.



- Kết quả:

## # 🛠️ Hướng dẫn Hiện thực hóa Mã Kiểm thử Chuyên sâu cho FR-02

Vì dự án dùng chuẩn `"type": "module"`, chúng ta sẽ viết mã bằng cú pháp ES Modules (`import/export`). Nhóm hãy viết tệp kiểm thử này để hiểu cách Playwright kiểm soát và đánh giá chất lượng của trang Login:

```

```javascript
import { test, expect } from '@playwright/test';

test.describe('FR-02: Học sâu Cơ chế Kiểm thử Đăng nhập & An toàn Biểu mẫu', () => {

  // Trước mỗi test case, luôn đưa trình duyệt về trang Đăng nhập
  test.beforeEach(async ({ page }) => {
    await page.goto('/login');
  });

  /**
   * BÀI HỌC 1: Kiểm thử Hộp xám vạch trần lỗi thiết kế Form (GUI Audit)
   * Mục tiêu: Ép Playwright phải kiểm tra các thuộc tính kỹ thuật của DOM để xem có đúng SRS không.
   */
  test('KT-01: Kiểm tra các thuộc tính an toàn bảo mật của biểu mẫu', async ({ page }) => {
    // Tìm ô nhập Email (Username)
    const emailField = page.locator('input').first();

    // ĐẶC TẢ: Bắt buộc dùng type="email" -> THỰC TẾ SUT: type="text"
    // Đoạn code này kiểm tra xem thuộc tính type có phải là "email" không. Nó sẽ FAIL vì lập trình viên làm sai.
    await expect(emailField).toHaveAttribute('type', 'email');

    // Tìm ô nhập Mật khẩu
    const passwordField = page.locator('input').nth(1);

    // ĐẶC TẢ: Bắt buộc dùng type="password" -> THỰC TẾ SUT: type="text"
    // Đoạn code này sẽ FAIL để vạch trần lỗi lộ plaintext mật khẩu trên giao diện!
    await expect(passwordField).toHaveAttribute('type', 'password');
  });

  /**
   * BÀI HỌC 2: Kiểm thử Vị trí của Thông báo lỗi (DOM Position Verification)
   * Mục tiêu: Xác minh yêu cầu FR-22 "Thông báo lỗi phải xuất hiện TRÊN nút submit, không phải bên dưới."
   */
  test('KT-02: Xác minh vị trí hiển thị của thông báo lỗi so với nút Submit', async ({ page }) => {

```

```

const emailField = page.locator('input').first();
const passwordField = page.locator('input').nth(1);
const submitButton = page.locator('button:has-text("Sign In)");
// Từ Login.jsx

// Cố tình nhập sai để kích hoạt thông báo lỗi
await emailField.fill('invalid_user@eshop.com');
await passwordField.fill('WrongPass123');
await submitButton.click();

// Khởi div báo lỗi hiển thị
const errorAlert = page.locator('.bg-red-100');
await expect(errorAlert).toBeVisible();

// Sử dụng JavaScript chạy ngầm trên trình duyệt để kiểm tra vị
trí hình học/DOM của 2 phần tử
const isErrorBelowButton = await page.evaluate(() => {
  const btn = document.querySelector('button[type="submit"]');
  const err = document.querySelector('.bg-red-100');
  if (!btn || !err) return false;
  // Trả về true nếu nút submit nằm TRƯỚC khối lỗi (tức là khối
lỗi nằm DƯỚI nút bấm)
  return !(btn.compareDocumentPosition(err) &
Node.DOCUMENT_POSITION_FOLLOWING);
});

// Đặc tả bắt lỗi phải nằm TRÊN, nên nếu err nằm DƯỚI
(isErrorBelowButton === true),
// câu khẳng định này sẽ bắn ra lỗi để đánh dấu SUT vi phạm tài
liệu thiết kế!
expect(isErrorBelowButton).toBe(false);
});

/**
 * BÀI HỌC 3: Kiểm thử Trạng thái Khóa Tài khoản (State Lockout
Machine)
 * Mục tiêu: Kiểm tra bộ đếm tăng đúng 1 đơn vị và khóa khi đủ 3 lần
liên tiếp.
 */
test('KT-03: Nhập sai liên tiếp 3 lần xem hệ thống có kích hoạt chặn
truy cập', async ({ page }) => {
  const emailField = page.locator('input').first();
  const passwordField = page.locator('input').nth(1);
  const submitButton = page.locator('button:has-text("Sign In)");
  const errorAlert = page.locator('.bg-red-100');

  // Chạy vòng lặp 3 lần nhập sai liên tiếp để kích hoạt điều kiện
khóa của FR-02
  for (let i = 1; i <= 3; i++) {
    await emailField.fill('lockout_test@eshop.com');
    await passwordField.fill(`WrongPass_Attempt_${i}`);
    await submitButton.click();

    // Kiểm tra thông báo lỗi xuất hiện ở mỗi lần

```

```

        await expect(errorAlert).toBeVisible();
        await expect(errorAlert).toHaveText('Đăng nhập thất bại. Vui
lòng kiểm tra lại.');
```

```

    }

    // ĐẶC TẢ: "Hệ thống trả về thông báo lỗi phù hợp; không để lộ chi
tiết nguyên nhân."
    // Nhóm hãy kiểm tra xem chuỗi thông báo lúc này có thay đổi để
cảnh báo người dùng đã bị khóa hay không,
    // hay nó vẫn hiển thị một câu thông báo tù mù giống hệt các lần
trước.
    await expect(errorAlert).toContainText('thất bại');
  });
});

```

- Nhận xét: Đây là một kịch bản kiểm thử đầu tiên của FR02 mà sinh viên đã tìm hiểu với sự hỗ trợ của A.I dựa trên mã nguồn, đây cũng chưa phải là bản hoàn thiện đầu tiên mà chỉ là bản nháp để học và hiểu sâu những tính năng cơ bản mà Playwright đã cung cấp nên kịch bản này vẫn chưa được gọi là tốt. Phiên bản này A.I đang sử dụng là CSS/XPath cho nên sẽ bị flaky (không ổn định) trong thời gian dài. Thông qua bản code này thì sinh viên nhận xét là mình không cần phải viết 100% bằng tay mà có thể sử dụng tính năng CodeGen để mà sinh ra Prototype đầu tiên với thuộc tính GetByRole hoặc bổ sung data-testid, sử dụng những biểu tượng như Assert Text, Assert Visibility, Assert Value rồi sau đó mình sẽ refactor kịch bản script dần dần.

## 23127396 – Lương Linh Khôi

### ChatGPT, OpenAI. Truy cập 6h15 ngày 11/07/2026

- Mục đích:

Hỗ trợ xây dựng một Journey kiểm thử chức năng **Đăng nhập (Login)** trên nền tảng Mabl, bao gồm các bước thực hiện, điều kiện kiểm tra (Assertions) và kết quả mong đợi để phục vụ việc tạo kịch bản kiểm thử End-to-End.

- Prompt:

Tôi đang sử dụng công cụ Mabl để xây dựng một Journey kiểm thử chức năng đăng nhập của một ứng dụng Web.

Hãy tạo một kịch bản kiểm thử End-to-End (E2E) cho chức năng Login với các yêu cầu sau:

1. Truy cập vào trang đăng nhập.
2. Nhập tên đăng nhập hợp lệ.
3. Nhập mật khẩu hợp lệ.
4. Nhấn nút "Login".
5. Kiểm tra người dùng được chuyển đến trang Dashboard sau khi đăng nhập thành công.
6. Kiểm tra thông báo chào mừng hoặc tên người dùng được hiển thị trên giao diện.

Đối với mỗi bước, hãy trình bày theo định dạng sau:

- Step
- Action
- Expected Result
- Assertion

Trả lời dưới dạng bảng để có thể dễ dàng chuyển thành một Journey trên Mabl.

- Kết quả:

AI sinh ra một kịch bản kiểm thử gồm đầy đủ các bước thao tác từ truy cập trang đăng nhập, nhập thông tin tài khoản, thực hiện đăng nhập đến các bước xác nhận sau khi đăng nhập thành công. Kịch bản cũng bao gồm các điều kiện kiểm tra (Assertions) và kết quả mong đợi tương ứng cho từng bước, giúp thuận tiện khi xây dựng Journey trên Mabl.

- Nhận xét:

Prompt giúp tạo nhanh cấu trúc ban đầu của Journey, giảm thời gian thiết kế testcase và đảm bảo các bước kiểm thử được trình bày đầy đủ, logic. Tuy nhiên, nội dung do AI sinh ra vẫn cần được người thực hiện kiểm tra, chỉnh sửa và bổ sung để phù hợp với giao diện, luồng nghiệp vụ và yêu cầu thực tế của hệ thống cần kiểm thử trước khi triển khai trên Mabl.

### **ChatGPT, OpenAI. Truy cập 5h35 ngày 11/07/2026**

- Mục đích:

Hỗ trợ tìm hiểu quy trình cài đặt và thiết lập môi trường làm việc của công cụ Mabl để phục vụ kiểm thử tự động (Automated Testing) cho ứng dụng Web.

- Prompt:

Tôi muốn tìm hiểu cách cài đặt và thiết lập công cụ Mabl để phục vụ kiểm thử tự động cho ứng dụng Web.

Hãy hướng dẫn chi tiết các nội dung sau:

1. Giới thiệu tổng quan về Mabl và các thành phần chính của công cụ.
2. Các yêu cầu cần chuẩn bị trước khi sử dụng Mabl.
3. Cách tạo tài khoản Mabl và tạo Workspace.
4. Cách tạo Application và cấu hình môi trường kiểm thử.
5. Cách cài đặt Mabl Trainer (nếu cần) và kết nối với Workspace.
6. Cách tạo Journey đầu tiên bằng tính năng Record.
7. Cách thực thi (Run) Journey và xem báo cáo kết quả kiểm thử (Run Results).
8. Những lỗi thường gặp trong quá trình cài đặt và cách khắc phục.

Hãy trình bày theo từng bước rõ ràng để người mới bắt đầu có thể làm theo.

- Kết quả:

ChatGPT cung cấp hướng dẫn từng bước về quá trình cài đặt và thiết lập Mabl, bao gồm việc tạo tài khoản, khởi tạo Workspace và Application, chuẩn bị môi trường kiểm thử, tạo Journey đầu tiên, thực thi kiểm thử và theo dõi báo cáo Run Results. Nội dung được trình bày theo trình tự hợp lý, giúp người mới dễ dàng tiếp cận và thực hành.

- Nhận xét:

Prompt giúp tổng hợp nhanh các bước cần thiết để bắt đầu sử dụng Mabl mà không cần tra cứu từ nhiều nguồn khác nhau. Tuy nhiên, do Mabl thường xuyên cập nhật giao diện và tính năng, người sử dụng vẫn cần đối chiếu với tài liệu chính thức của Mabl để đảm bảo các bước cài đặt và cấu hình luôn phù hợp với phiên bản mới nhất của công cụ.

## 23127449 – Nguyễn Tấn Phát

- Claude Sonnet 4.6, Anthropic, 2026-07-09 01:23:

- Prompt: (Tạo các skill **wat-\***)

Tôi đang sử dụng công cụ Mabl để xây dựng một Journey kiểm thử chức năng đăng nhập của một ứng dụng Web.

Hãy tạo một kịch bản kiểm thử End-to-End (E2E) cho chức năng Login với các yêu cầu sau:

1. Truy cập vào trang đăng nhập.
2. Nhập tên đăng nhập hợp lệ.
3. Nhập mật khẩu hợp lệ.
4. Nhấn nút "Login".
5. Kiểm tra người dùng được chuyển đến trang Dashboard sau khi đăng nhập thành công.
6. Kiểm tra thông báo chào mừng hoặc tên người dùng được hiển thị trên giao diện.

Đối với mỗi bước, hãy trình bày theo định dạng sau:

- Step
- Action
- Expected Result
- Assertion

Trả lời dưới dạng bảng để có thể dễ dàng chuyển thành một Journey trên Mabl.

– Kết quả:

AI sinh ra một kịch bản kiểm thử gồm đầy đủ các bước thao tác từ truy cập trang đăng nhập, nhập thông tin tài khoản, thực hiện đăng nhập đến các bước xác nhận sau khi đăng nhập thành công. Kịch bản cũng bao gồm các điều kiện kiểm tra (Assertions) và kết quả mong đợi tương ứng cho từng bước, giúp

thuận tiện khi xây dựng Journey trên Mabl.

– Nhận xét:

Prompt giúp tạo nhanh cấu trúc ban đầu của Journey, giảm thời gian thiết kế testcase và đảm bảo các bước kiểm thử được trình bày đầy đủ, logic. Tuy nhiên, nội dung do AI sinh ra vẫn cần được người thực hiện kiểm tra, chỉnh sửa và bổ sung để phù hợp với giao diện, luồng nghiệp vụ và yêu cầu thực tế của hệ thống cần kiểm thử trước khi triển khai trên Mabl.

**\*\*ChatGPT, OpenAI. Truy cập 5h35 ngày 11/07/2026\*\***

– Mục đích:

Hỗ trợ tìm hiểu quy trình cài đặt và thiết lập môi trường làm việc của công cụ Mabl để phục vụ kiểm thử tự động (Automated Testing) cho ứng dụng Web.

– Prompt:

Tôi muốn tìm hiểu cách cài đặt và thiết lập công cụ Mabl để phục vụ kiểm thử tự động cho ứng dụng Web.

Hãy hướng dẫn chi tiết các nội dung sau:

1. Giới thiệu tổng quan về Mabl và các thành phần chính của công cụ.
2. Các yêu cầu cần chuẩn bị trước khi sử dụng Mabl.
3. Cách tạo tài khoản Mabl và tạo Workspace.
4. Cách tạo Application và cấu hình môi trường kiểm thử.
5. Cách cài đặt Mabl Trainer (nếu cần) và kết nối với Workspace.
6. Cách tạo Journey đầu tiên bằng tính năng Record.
7. Cách thực thi (Run) Journey và xem báo cáo kết quả kiểm thử (Run Results).
8. Những lỗi thường gặp trong quá trình cài đặt và cách khắc phục.

Hãy trình bày theo từng bước rõ ràng để người mới bắt đầu có thể làm theo.

– Kết quả:

ChatGPT cung cấp hướng dẫn từng bước về quá trình cài đặt và thiết lập Mabl, bao gồm việc tạo tài khoản, khởi tạo Workspace và Application, chuẩn bị môi trường kiểm thử, tạo Journey đầu tiên, thực thi kiểm thử và theo dõi báo cáo Run Results. Nội dung được trình bày theo trình tự hợp lý, giúp người mới dễ dàng tiếp cận và thực hành.

– Nhận xét:

Prompt giúp tổng hợp nhanh các bước cần thiết để bắt đầu sử dụng Mabl mà không cần tra cứu từ nhiều nguồn khác nhau. Tuy nhiên, do Mabl thường xuyên cập nhật giao diện và tính năng, người sử dụng vẫn cần đối chiếu với tài liệu chính thức của Mabl để đảm bảo các bước cài đặt và cấu hình luôn phù

hợp với phiên bản mới nhất của công cụ.

**\*\*23127449 – Nguyễn Tấn Phát\*\***

– Claude Sonnet 4.6, Anthropic, 2026-07-09 01:23:

– **\*\*Prompt: (Tạo các skill `wat-\*`)\*\***

```

Tiếp theo hãy dùng skill skill-creator để tạo skill <Tên Skill> cho tôi. Bạn cần tham khảo nội dung và workflow đã chốt về skill này trong AGENTS.md để đảm bảo tạo đúng nhất. Tôi có các lưu ý về skill này như sau (nội dung skill phải bằng tiếng anh):

- Bạn phải chỉ rõ ra input, output là gì (nếu có)
  - Workflow chi tiết các bước để hướng dẫn cho AI biết trong skill này cần làm những gì, human verification gate ở các chỗ nào
  - Tiêu chí nào để đánh giá hoàn thành trước khi kết thúc
  - Quality checklist cho các kết quả được tạo ra trong skill này, và cho cả workflow trong skill
  - Trong skill này được phép làm những gì, không được làm những gì
  - Các nội dung bạn cung cấp phải đúng chuẩn các best practice, không được bịa, phải dựa trên các lý thuyết thực tế
  - Đảm bảo khả năng reusable của skill phải cao, để nó có thể ứng dụng vào được mọi project khi cần thực hiện web automation testing
  - Và các nội dung cần thiết khác liên quan đến skill này
- ```

– **\*\*AI Output:\*\*** Các skill `wat-\*` trong `.agents/skills/` của repo [phatnguyen975/eshop-sut](https://github.com/phatnguyen975/eshop-sut/tree/web-automation-testing)

– Claude Sonnet 4.6, Anthropic, 2026-07-08 10:45:

– **\*\*Prompt: (Tạo skill `functional-test-design`, skill này gồm các skill độc lập nhau gộp lại như Domain Testing, Decision Table Testing, ... Prompt này tạo từng sub-skill độc lập sau đó gộp lại thành skill lớn)\*\***

```

Tôi đang muốn tạo một skill về kỹ thuật <Tên Skill>. Tôi có các yêu cầu về skill này như sau:

- Lưu ý skill này là kỹ thuật để thiết kế chứ không phải dùng để viết code.
- Nội dung của skill trong SKILL.md phải có ít nhất các mục sau:
  - Overview: Giới thiệu tổng quát về chức năng và mục đích skill này
  - When to Use: Khi nào thì skill này sẽ được dùng
  - When NOT to Use: Khi nào thì skill này không được dùng
  - Input nếu có: các input mà skill này cần có
  - Core Principles: Các nguyên tắc/rules chính của skill này (bạn cần đánh giá xem mục này có cần hay không, hay đổi thành mục khác, do mục này optional)
  - Design Process: Các bước để thiết kế test case (phải tạo thành một quy trình hoàn chỉnh các bước nhỏ nhất như xác định các requirements, BR,



constrainst, ...), đảm bảo đúng theo lý thuyết nhất, tên section này bạn có thể đổi lại sao cho hợp lý nhất. Và phải luôn có bước để AI review lại các quality checklist để xem coi đã thiết kế đúng và đủ nhất hay chưa.

- Design Rules (nếu có)
- Anti-Patterns: các anti patterns không được làm theo
- Best practices: các best practice nên làm theo
- Quality checklist (khác với checklist trong Design Process):

checklist này dùng để kiểm tra xem có tuân thủ đúng các rule, đúng các best practices, đúng và đủ các steps, ... hay chưa, còn checklist kia dùng để kiểm tra chất lượng của các test case đã thiết kế xem có đúng và đủ hay chưa. Do đó bạn cần phân tích để đặt tên lại cho đúng

- Common Rationalizations
- Red Flags (nếu cần)
- Output nếu có: các output mà skill này cần có
- Và các mục khác nếu bạn thấy skill này cần đến, hoặc có thể bỏ bớt

đi các mục trên mà tôi liệt kê nếu không cần

- Lưu ý trong SKILL.md bạn không cần trình bày chi tiết nhất cho các nội dung trên, mà có thể link tới các file trong resources/ hay examples/ và trong các file này sẽ cần trình bày chi tiết. Mục đích để giữ cho SKILL.md không quá dài, tránh gây tình trạng AI đọc thiếu

- Và cần có ít nhất một ví dụ mà bạn áp dụng skill này (kỹ thuật này) và để thiết kế ra các test case đúng và đủ nhất cho trường hợp đó. Ví dụ bạn có thể đưa ra một BR hay một mô tả cho feature cụ thể (FR cụ thể) và tiến hành đi phân tích BR, ... và thiết kế cho đúng nhất. Phải đảm bảo đủ, không thừa, không thiếu.

- Skill phải đảm bảo có thể reuse được cho nhiều project, các nội dung phải đúng theo lý thuyết thực tế, không được bịa, từ ngữ sử dụng chuẩn production.

```

- **\*\*AI Output:\*\*** Skill `functional-test-design` trong `.agents/skills/` của repo [phatnguyen975/eshop-sut](https://github.com/phatnguyen975/eshop-sut/tree/web-automation-testing)

- Claude Sonnet 4.6, Anthropic, 2026-07-10 05:37:

- **\*\*Prompt:** (Tạo skill `scenario-test-design`)\*\*

```

Tôi đang muốn tạo một skill về kỹ thuật scenario testing để thiết kế các scenario flow, skill này dùng để hỗ trợ tạo ra các kịch bản test (e2e flow) trước khi tạo các data cho từng kịch bản. Hãy dùng skill skill-creator để tạo skill scenario-testing cho tôi.

Tôi có các yêu cầu về skill này như sau:

- Lưu ý skill này là kỹ thuật để thiết kế chứ không phải dùng để viết code.

- Tôi có gửi kèm cho bạn file scenario-testing.md, trong này chứa các lý thuyết mà tôi đã được học về kỹ thuật này. Gồm 16 best practice để tạo một good scenario. Bạn cần tìm hiểu và phân tích thêm các lý thuyết trong thực tế bên cạnh các lý thuyết này để đảm bảo nội dung cung cấp cần đầy đủ nhất, và phải có Design Process rõ ràng các bước (hiện tại file này chưa có). Và phải có thêm mode khi invoke để có thể lưu ra file hoặc in ra

conversation trực tiếp theo như mô tả trong file này.

- Nội dung của skill trong SKILL.md phải có ít nhất các mục sau:
  - Overview: Giới thiệu tổng quát về chức năng và mục đích skill này
  - When to Use: Khi nào thì skill này sẽ được dùng
  - When NOT to Use: Khi nào thì skill này không được dùng
  - Input nếu có: các input mà skill này cần có
  - Core Principles: Các nguyên tắc/rules chính của skill này (bạn cần đánh giá xem mục này có cần hay không, hay đổi thành mục khác, do mục này optional)
  - Design Process: Các bước để thiết kế test case (phải tạo thành một quy trình hoàn chỉnh các bước nhỏ nhất như xác định các requirements, BR, constraint, ...), đảm bảo đúng theo lý thuyết nhất, tên section này bạn có thể đổi lại sao cho hợp lý nhất. Và phải luôn có bước để AI review lại các quality checklist để xem coi đã thiết kế đúng và đủ nhất hay chưa.
  - Design Rules (nếu có)
    - Anti-Patterns: các anti patterns không được làm theo
    - Best practices: các best practice nên làm theo
    - Quality checklist (khác với checklist trong Design Process): checklist này dùng để kiểm tra xem có tuân thủ đúng các rule, đúng các best practices, đúng và đủ các steps, ... hay chưa, còn checklist kia dùng để kiểm tra chất lượng của các test case đã thiết kế xem có đúng và đủ hay chưa. Do đó bạn cần phân tích để đặt tên lại cho đúng
  - Common Rationalizations
  - Red Flags (nếu cần)
  - Output nếu có: các output mà skill này cần có
  - Và các mục khác nếu bạn thấy skill này cần đến, hoặc có thể bỏ bớt đi các mục trên mà tôi liệt kê nếu không cần
- Lưu ý trong SKILL.md bạn không cần trình bày chi tiết nhất cho các nội dung trên, mà có thể link tới các file trong resources/ hay examples/ và trong các file này sẽ cần trình bày chi tiết. Mục đích để giữ cho SKILL.md không quá dài, tránh gây tình trạng AI đọc thiếu
- Và cần có ít nhất một ví dụ mà bạn áp dụng skill này (kỹ thuật này) và để thiết kế ra các test case đúng và đủ nhất cho trường hợp đó. Ví dụ bạn có thể đưa ra một BR hay một mô tả cho feature cụ thể (FR cụ thể) và tiến hành đi phân tích BR, ... và thiết kế cho đúng nhất. Phải đảm bảo đủ, không thừa, không thiếu.
- Skill phải đảm bảo có thể reuse được cho nhiều project, các nội dung phải đúng theo lý thuyết thực tế, không được bịa, từ ngữ sử dụng chuẩn production.

```

- **\*\*AI Output:\*\*** Skill `scenario-test-design` trong `.agents/skills/` của repo [phatnguyen975/eshop-sut](https://github.com/phatnguyen975/eshop-sut/tree/web-automation-testing)

# Công việc dự định cho tuần sau

**\*\*23127378 – Nguyễn Gia Huy\*\***

- Chuẩn bị nội dung cho Mabl User Guide
- Làm video demo/docs trình bày cho cả nhóm (nếu có thời gian)
- Tìm hiểu vài giải pháp AI test automation khác

**\*\*23127379 – Thái Minh Huy\*\***

- Thiết kế kịch bản End-to-end test với lý thuyết của Functional Testing,

Scenario Testing để thử nghiệm kịch bản phù hợp cho Seminar

- Tìm hiểu Skill liên quan đến Playwright để xây dựng workflow cho Playwright
- Chuẩn bị nội dung cho User Guide và Activity Worksheet cho buổi Seminar

**\*\*23127396 – Lương Linh Khôi\*\***

- \* Tiếp tục tìm hiểu và thực hành sử dụng Mabl.
- \* Xây dựng các Journey kiểm thử cho các chức năng được phân công.
- \* Chạy và đánh giá kết quả kiểm thử, điều chỉnh Journey khi cần.
- \* Chuẩn bị tài liệu và artifact phục vụ buổi Seminar.

**\*\*23127449 – Nguyễn Tấn Phát\*\***

- Tiến hành tạo các kịch bản kiểm thử, viết script dựa trên các kịch bản được phân công, chạy script để kiểm tra SUT. Toàn bộ quá trình này sẽ có sự hỗ trợ của AI dựa trên các skill và workflow đã được setup trước đó.

#### # Vấn đề

Hiện tại nhóm chưa có vấn đề thêm, tuy nhiên, nhóm sẽ tiếp tục thực hiện nghiên cứu sâu và cập nhật nếu có vấn đề phát sinh trong quá trình thực hiện các công việc dự định cho tuần sau.